SERVICIO NACIONAL DE APRENDIZAJE - SENA

TECNOLOGÍA EN ANÁLISIS Y DESARROLLO DE SOFTWARE

HOTEL GRAN MARQUES

Sistema de Gestión Hotelera

EVIDENCIA GA8-220501096-AA1-EV02

MÓDULOS INTEGRADOS

APRENDIZ

ALEJANDRO RAMÍREZ HERRERA

Identificación: C.C. 1036966567

Programa: Tecnología en Análisis y Desarrollo de Software (ADSO)

Ficha: 3235881

Proyecto: Sistema de Gestión Hotelera - Hotel Gran Marques

Fecha: 30 de octubre de 2026

Informe de entrega de módulos, componentes, pruebas, configuración y manual técnico

Tabla de contenido

1. Introducción

2. Objetivo y alcance de la entrega

3. Descripción del proyecto y requisitos

4. Inventario de módulos integrados

5. Arquitectura y componentes

6. Documentación de entradas y salidas

7. Seguridad, reglas de negocio y persistencia

8. Configuración de servidor y almacenamiento

9. Ambientes de desarrollo, pruebas y distribución

10. Pruebas realizadas y resultados

11. Código fuente, archivos de distribución y versionamiento

12. URLs y estado de despliegue

13. Acta de revisión y aprobación de requisitos

14. Manual técnico resumido

15. Conclusiones

16. Referencias

Nota: los módulos se documentan sobre el código fuente integrado incluido en esta carpeta. La plantilla de acta se entrega para revisión y firma real de los interesados; no se presenta como aprobada.

1. Introducción

Esta evidencia documenta la entrega de los módulos integrados del Sistema de Gestión Hotelera del Hotel Gran Marques. El propósito es presentar el código fuente, la distribución ejecutable de la aplicación web, la organización de los componentes, los datos que recibe y entrega cada módulo, los resultados de pruebas, la configuración de los ambientes y el manual técnico necesario para instalar y operar la solución.

La integración articula una interfaz construida con HTML, CSS y JavaScript, servicios cliente que consumen una API REST en Node.js, reglas de negocio y persistencia local en archivos JSON. El sistema reúne autenticación, huéspedes, habitaciones, reservas, servicios adicionales, facturación, pagos, mantenimiento y reportes. Las solicitudes del cliente se procesan en el servidor y las respuestas estructuradas se reflejan en los componentes de interfaz.

2. Objetivo y alcance de la entrega

Objetivo general: entregar los módulos del sistema integrados, documentados y probados, con instrucciones para reproducir el ambiente de desarrollo y pruebas, junto con los archivos necesarios para ejecutar la aplicación.

- Entregar el código fuente organizado por capas y componentes reutilizables.
- Incluir la carpeta de distribución `dist/` generada para ejecución local.
- Documentar funciones, entradas, salidas, rutas y validaciones de cada módulo.
- Presentar los resultados comprobables de las pruebas unitarias y de integración.
- Describir configuración de Node.js, variables de entorno y almacenamiento JSON.
- Incluir manual técnico, diagramas y referencia al repositorio Git del proyecto.

El alcance corresponde a una versión académica local. La persistencia es JSON y no existe una URL pública de hosting suministrada para esta entrega. Para operación productiva se requerirían base de datos transaccional, HTTPS, almacenamiento de secretos, monitoreo y despliegue en un servidor administrado.

3. Descripción del proyecto y requisitos

Hotel Gran Marques es una aplicación web orientada a apoyar las operaciones internas de un establecimiento hotelero. Los perfiles principales son administración, recepción y gerencia. El flujo inicia con autenticación y habilita los módulos de acuerdo con el rol asignado.

3.1 Requisitos funcionales consolidados

ID	Módulo	Requisito
RF-01	Autenticación	Iniciar sesión, registrar cuenta y consultar perfil.
RF-02	Huéspedes	Crear, listar, buscar, consultar, actualizar y eliminar según reglas.
RF-03	Habitaciones	Administrar catálogo, capacidad, tipo, tarifa y disponibilidad.
RF-04	Reservas	Crear, consultar, modificar y controlar disponibilidad por fechas.
RF-05	Servicios	Consultar catálogo y anexar consumos a una reserva.
RF-06	Facturación	Generar y consultar facturas asociadas a reservas.
RF-07	Pagos	Registrar abonos, método y referencia; calcular saldo.
RF-08	Mantenimiento	Crear solicitudes y actualizar prioridad/estado.
RF-09	Reportes	Consultar indicadores para administración y gerencia.

3.2 Requisitos no funcionales

- Seguridad: autenticación, JWT, hash de contraseñas y autorización por rol.
- Integridad: validación de campos, relaciones entre registros, fechas y saldos.
- Mantenibilidad: separar presentación, cliente API, adaptadores, servicios y servidor.
- Usabilidad: navegación consistente, formularios con etiquetas y mensajes de resultado.
- Portabilidad: ejecución en equipos con Node.js 18 o superior y navegador moderno.
- Trazabilidad: control de cambios en Git y documentación de pruebas.

3.3 Casos de uso representativos

- UC-01 Iniciar sesión: recepción o administración ingresa credenciales; el servidor valida y habilita los módulos permitidos.
- UC-02 Registrar huésped: recepción diligencia datos de identificación y contacto; el sistema valida campos y evita documentos duplicados.
- UC-03 Crear reserva: recepción selecciona huésped, habitación y fechas; el sistema valida capacidad, disponibilidad y cruces de fechas.
- UC-04 Facturar y recibir pago: el sistema genera la factura asociada a la reserva y registra abonos sin superar el saldo.

4. Inventario de módulos integrados

Módulo	Funcionalidad integrada	Componentes principales
Autenticación	Login, registro, sesión, perfil y cierre de sesión.	auth-service.js / api-client.js / api/server.js
Huéspedes	Altas, consulta, búsqueda, edición y eliminación condicionada.	hotel-service.js / rutas /api/guests
Habitaciones	Catálogo, disponibilidad, tipo, tarifa y estado.	hotel-service.js / rutas /api/rooms
Reservas	Creación, consulta, actualización, cancelación y consumos.	hotel-service.js / rutas /api/reservations
Servicios	Catálogo de servicios y consumos por reserva.	hotel-service.js / rutas /api/services
Facturación	Creación y consulta de comprobantes.	hotel-service.js / rutas /api/invoices
Pagos	Registro de abonos y cálculo de saldo.	hotel-service.js / rutas /api/invoices/{id}/payments
Mantenimiento	Solicitudes, prioridad y seguimiento de estado.	hotel-service.js / rutas /api/maintenance
Reportes	Indicadores operativos y gerenciales.	hotel-service.js / ruta /api/reports/summary

La relación de módulos retoma el dominio hotelero descrito en los antecedentes del proyecto: huéspedes, reservas, habitaciones, facturas, recepción, mantenimiento, áreas sociales e informes. La implementación presentada en este paquete operacionaliza esos procesos mediante vistas, servicios cliente y rutas REST.

4.1 Navegación entre módulos

- La pantalla de login es el punto de entrada.
- Después de autenticarse se muestra el dashboard.
- El menú principal permite acceder a huéspedes, habitaciones, reservas, servicios, facturación, mantenimiento y reportes.
- Las acciones de creación/edición se realizan desde el módulo correspondiente y regresan al listado o panel.
- Los permisos del backend complementan el control visual de opciones por rol.

5. Arquitectura y componentes

La solución sigue una separación por capas: presentación web, integración cliente, servicios/API, dominio y persistencia. La separación reduce acoplamiento y permite cambiar la interfaz o el almacenamiento sin mezclar todas las responsabilidades en un único archivo.

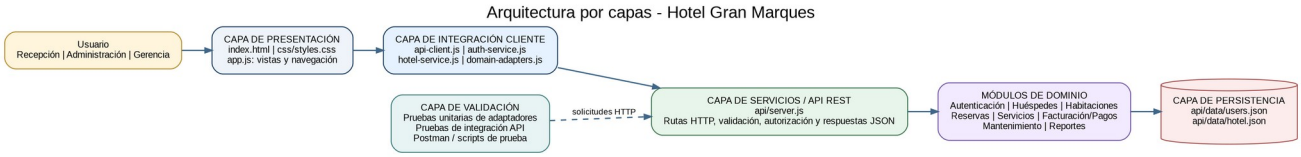


Figura 1. Arquitectura por capas del Sistema de Gestión Hotelera.

5.1 Paquetes principales

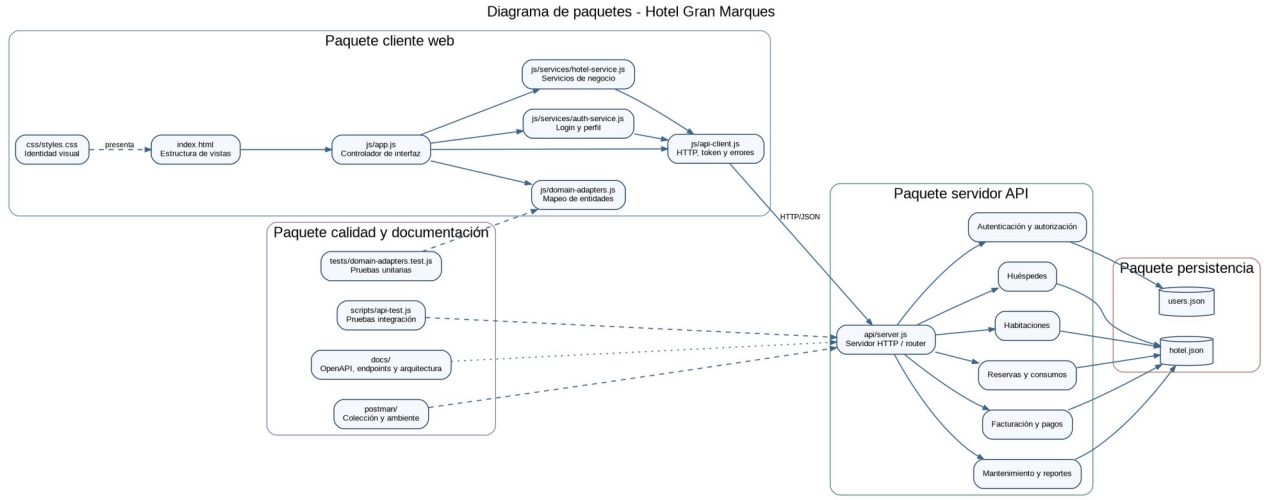


Figura 2. Organización de paquetes del cliente, servidor, persistencia y calidad.

5.2 Componentes y dependencias

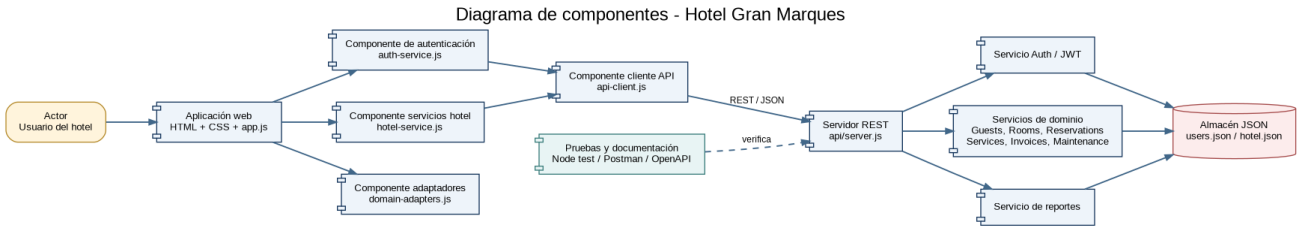


Figura 3. Componentes y comunicaciones HTTP/JSON.

El cliente utiliza `api-client.js` como punto central de comunicación HTTP. `auth-service.js` atiende la sesión y `hotel-service.js` agrupa operaciones del dominio. `domain-adapters.js` adapta los modelos para las vistas. En el servidor, `api/server.js` recibe solicitudes, aplica validaciones y autorización, ejecuta reglas de dominio y persiste los cambios.

6. Documentación de entradas y salidas por módulo

Módulo	Entradas	Salidas	Errores esperados
Autenticación	Registro: name, username, email, password. Login: username, password. Perfil: token Bearer.	Registro devuelve user y 201; login devuelve user + token y 200; perfil devuelve datos públicos.	400, 401, 409
Huéspedes	name, document, email, phone, nationality; búsqueda opcional q.	ID, ficha del huésped, listado y conteo.	400, 401, 404, 409
Habitaciones	number, type, capacity, rate, status, description; filtros status/type.	ID, ficha de habitación, tarifa, capacidad, disponibilidad.	400, 401, 403, 404, 409
Reservas	guestId, roomId, checkIn, checkOut, guestsCount, notes.	ID/código, noches, total, habitación, huésped y estado.	400, 401, 404, 409
Servicios	name, description, price, active; consumo: serviceId, quantity.	Catálogo o reserva actualizada con consumos y subtotales.	400, 401, 403, 404
Facturas	reservationId.	Número, conceptos, total, estado de pago, pagos y saldo.	400, 401, 404, 409
Pagos	amount, method, reference.	Registro de abono, total pagado y saldo.	400, 401, 404, 409
Mantenimiento	roomId, area, description, priority; cambios de status/notes.	Solicitud identificada con estado, prioridad y descripción.	400, 401, 404
Reportes	Solicitud autenticada; sin payload obligatorio.	Indicadores de habitaciones, reservas, ocupación e ingresos.	401, 403

El detalle de estructuras JSON, obligatoriedad de campos y ejemplos de respuesta se encuentra en `docs/API_REFERENCE.md`, el inventario de rutas está en `docs/ENDPOINTS_API.txt` y el contrato de intercambio está descrito en `docs/openapi.json`.

6.1 Respuestas HTTP relevantes

Código	Significado	Ejemplo de uso
200	OK	Consulta, login o actualización exitosa.
201	Created	Creación de huésped, reserva, factura o solicitud.
400	Bad Request	Campos obligatorios o formatos no válidos.
401	Unauthorized	Token ausente/no válido o credenciales incorrectas.
403	Forbidden	El rol autenticado no puede ejecutar la operación.
404	Not Found	El recurso solicitado no existe.
409	Conflict	Duplicados, cruce de reservas o factura repetida.

7. Seguridad, reglas de negocio y persistencia

7.1 Mecanismos de seguridad

- Las contraseñas se almacenan mediante scrypt con sal aleatoria, no en texto plano.
- La autenticación satisfactoria emite un JWT firmado con HS256 y tiempo de vida limitado.
- Las rutas privadas reciben `Authorization: Bearer <token>` y validan firma y expiración.
- El backend verifica roles para operaciones administrativas, sin confiar solo en la interfaz.
- El secreto JWT debe almacenarse en `.env` local o en variables seguras del hosting y nunca publicarse.
- El archivo `.gitignore` evita versionar `.env` y datos JSON locales.

El uso de scrypt está alineado con la guía de OWASP para almacenamiento de contraseñas cuando Argon2id no se utiliza; para producción se deben revisar los parámetros de coste y la gestión de secretos (OWASP Foundation, s. f.).

7.2 Reglas de negocio e integridad

- El documento de un huésped y el número de habitación deben ser únicos.
- Una reserva exige un huésped y una habitación válidos, fechas coherentes y capacidad suficiente.
- No se admite solapamiento de reservas activas para la misma habitación.
- Una reserva no puede tener más de una factura.
- Los abonos no pueden exceder el saldo pendiente.
- La creación y modificación del catálogo de habitaciones y servicios exige rol administrativo.

7.3 Persistencia

La versión académica usa dos archivos JSON locales: `api/data/users.json` para cuentas y `api/data/hotel.json` para entidades operativas. El servidor los crea cuando inicia. Esta estrategia permite reproducir el proyecto sin instalar un motor externo. No ofrece transacciones multiusuario ni garantías de concurrencia propias de una base de datos transaccional; la siguiente evolución técnica recomendada es PostgreSQL o MySQL con repositorios y transacciones.

8. Configuración de servidor y almacenamiento

Elemento	Configuración académica	Observación
Runtime	Node.js 18 o superior	El aprendiz reportó Node.js v24.22.1 en Git Bash. La verificación de construcción se ejecutó con v22.16.0.
Servidor	HTTP nativo de Node.js en `api/server.js`	Puerto por defecto 3000.
Variables	PORT, NODE_ENV, JWT_SECRET	Definidas en `.env`; `.env.example` sirve como plantilla.
Interfaz	HTML5, CSS3, JavaScript	Servida desde la raíz de la aplicación.
Almacenamiento	JSON en `api/data/`	Se crea en primer inicio y se excluye de Git.
Dependencias	Módulos nativos de Node.js	`npm install` no requiere paquetes de terceros.
Contrato API	REST/JSON y `docs/openapi.json`	Respuestas uniformes y códigos HTTP.

8.1 Variables de entorno

```
PORT=3000
NODE_ENV=development
JWT_SECRET=REEMPLAZAR_POR_UN_SECRETO_ALEATORIO_DE_32_CARACTERES_O_MAS
```

El valor de ejemplo no se debe utilizar en producción. Cada ambiente debe contar con un secreto diferente y privado.

8.2 Comandos de instalación y ejecución

```
cp .env.example .env
# Editar .env y definir JWT_SECRET
node api/server.js
```

En el equipo del aprendiz, `node api/server.js` quedó ejecutando el servidor correctamente. Debido a una diferencia en el PATH que usan los procesos secundarios de Windows, `npm start` no reconoció `node` en esa terminal; por ello se documenta el comando directo que fue comprobado.

8.3 Verificación de salud

```
GET http://localhost:3000/api/health
Respuesta de referencia: { "success": true, "data": { "status": "ok" } }
```

9. Ambientes de desarrollo, pruebas y distribución

Ambiente	Finalidad	Configuración / URL
Desarrollo local	Codificación y validación manual.	Node.js, Git, navegador; `NODE_ENV=development`; `http://localhost:3000/`.
Pruebas local	Ejecutar pruebas unitarias y de integración.	Servidor local en puerto 3000; comandos Node descritos en sección 10.
Distribución	Paquete de ejecución para entregar.	Carpeta `dist/` creada por `node scripts/build.js`.
Producción	No está habilitado en el paquete actual.	Requiere hosting Node, HTTPS, variables secretas y base de datos administrada.

9.1 Archivos de distribución

El proyecto contiene `scripts/build.js`, que copia el HTML, CSS, JavaScript, la API, la plantilla de entorno y documentación técnica esencial a la carpeta `dist/`. Esta carpeta es el artefacto de distribución ejecutable para la plataforma Node.js; no se genera un archivo nativo `.exe` porque el sistema es una aplicación web interpretada por Node.js y un navegador.

- Código fuente: `index.html`, `css/`, `js/`, `api/`, `tests/`, `scripts/`.
- Distribución: `dist/` con frontend y servidor.
- Documentación: `docs/` con API, arquitectura, entradas/salidas, ambiente y resultados.
- Control de cambios: `.gitignore`, `VERSIONAMIENTO_GIT.md` y enlace al repositorio.

9.2 Construir la carpeta dist

```
node scripts/build.js
# La salida se crea en ./dist
```

Para iniciar la distribución, ubíquese en `dist/`, configure un `.env` local y ejecute `node api/server.js`.

10. Pruebas realizadas y resultados

Se ejecutaron pruebas unitarias sobre los adaptadores de dominio y pruebas de integración HTTP contra el servidor local. Las pruebas verifican tanto respuestas exitosas como restricciones de integridad y autorización. Los scripts reproducibles se entregan junto con el código.

10.1 Pruebas unitarias

Caso	Componente	Validación	Resultado
UT-01	mapRoom	Conserva identificador, número, capacidad y tarifa.	Aprobada
UT-02	mapGuest	Normaliza datos de huésped.	Aprobada
UT-03	mapReservation	Relaciona huésped/habitación, noches y consumos.	Aprobada
UT-04	mapService	Normaliza precio, descripción e indicador activo.	Aprobada
UT-05	mapInvoice	Calcula alojamiento, servicios, pagos y saldo.	Aprobada
UT-06	mapMaintenance	Resuelve ubicación desde la habitación.	Aprobada

Comando: `node --test tests/*.test.js`. Resultado ejecutado: 6 pruebas aprobadas y 0 fallidas.

10.2 Pruebas de integración

Grupo funcional	Escenarios verificados	Resultado
Servicio y sesión	Salud API, login admin, perfil JWT, registro, login de usuario y contraseña incorrecta.	Aprobado
Huéspedes	Crear, obtener identificador y buscar huésped.	Aprobado
Habitaciones	Consultar disponibilidad; validar permisos de recepción y alta con administrador.	Aprobado
Reservas	Crear, calcular noches, detectar cruce de fechas y consultar reserva.	Aprobado
Servicios	Consultar catálogo activo y asociar consumo a reserva.	Aprobado
Facturación y pagos	Crear factura, bloquear duplicado y registrar abono.	Aprobado
Mantenimiento y reportes	Crear y actualizar solicitud; consultar indicadores gerenciales.	Aprobado
Seguridad	Rechazar solicitud protegida sin token.	Aprobado

Comando: `node scripts/api-test.js`, con el servidor iniciado en otra terminal. Resultado ejecutado: 36 comprobaciones aprobadas y 0 fallidas.

10.3 Resultado consolidado

Tipo	Aprobadas	Fallidas	Total
Unitarias	6	0	6
Integración HTTP	36	0	36
TOTAL	42	0	42

Los resultados se reproducen en `docs/RESULTADOS_GA8_EV02.txt`, `docs/resultado_pruebas_integracion_GA8_EV02.json` y `docs/resultado_pruebas_integracion_GA8_EV02.txt`. Son pruebas automatizadas locales; no sustituyen aceptación formal del cliente, prueba de carga ni auditoría de seguridad productiva.

11. Código fuente, distribución y control de versiones

11.1 Estructura del proyecto

```
ALEJANDRO_RAMIREZ_GA8_EV02/
|-- index.html
|-- css/styles.css
|-- js/
|   |-- app.js
|   |-- api-client.js
|   |-- domain-adapters.js
|   `-- services/ (auth-service.js, hotel-service.js)
|-- api/server.js
|-- api/data/ (archivos JSON generados localmente)
|-- tests/domain-adapters.test.js
|-- scripts/ (api-test.js, build.js, smoke-test.js)
|-- docs/ (manuales, API, diagramas y pruebas)
|-- postman/
`-- dist/ (generado al construir)
```

11.2 Repositorio

Repositorio remoto indicado y utilizado en el proceso de versionamiento del proyecto:

```
https://github.com/Alejandroramirez07/Gran_marques_frontend
```

La URL anterior identifica el repositorio del proyecto. Para asegurar que esta entrega específica esté publicada, se debe copiar `ALEJANDRO_RAMIREZ_GA8_EV02/` a la raíz local del repositorio, realizar commit y push, y verificar la carpeta en la rama `main`.

```
git status
git add ALEJANDRO_RAMIREZ_GA8_EV02/
git commit -m "GA8-220501096-AA1-EV02: modulos integrados"
git push origin main
```

11.3 Trazabilidad de entregables

Producto	Ubicación	Finalidad
Código fuente	Raíz del paquete	Código completo de presentación, API y pruebas.
Distribución	`dist/`	Carpeta de salida ensamblada para ejecutar con Node.js.
Documentos	`docs/`	Manual, módulos, arquitectura, entradas/salidas, pruebas y acta.
Contrato API	`docs/openapi.json` y `docs/ENDPOINTS_API.txt`	Especificación de rutas y formatos de intercambio.
Repositorio	GitHub indicado arriba	Historial de cambios y respaldo remoto.

12. URLs y estado de despliegue

Recurso	URL	Estado / condición
Interfaz web local	http://localhost:3000/	Disponible mientras el servidor esté iniciado en el equipo local.
Raíz API local	http://localhost:3000/api	Disponible en el equipo local.
Salud API	http://localhost:3000/api/health	Endpoint verificado con HTTP 200.
Repositorio GitHub	https://github.com/Alejandroramirez07/Gran_marques_frontend	Repositorio del proyecto; confirmar publicación de esta carpeta EV02 después del push.
Despliegue público	No suministrado	Pendiente. No se declara una URL de hosting no verificada.

Las direcciones `localhost` son de desarrollo/pruebas y no pueden ser accedidas desde internet por el evaluador. Para satisfacer el requisito de URL desplegada, el aprendiz debe publicar la aplicación en un hosting compatible con Node.js, configurar variables de entorno y persistencia, y agregar aquí la URL HTTPS real una vez validada.

12.1 Recomendaciones para despliegue

- Usar un hosting que admita procesos Node.js persistentes.
- Configurar `PORT` según lo suministre la plataforma y almacenar `JWT_SECRET` como secreto, no en el repositorio.
- Cambiar credenciales de demostración y habilitar HTTPS.
- Sustituir JSON local por una base de datos administrada con copias de seguridad.
- Ejecutar la batería de integración contra la URL publicada y documentar resultados.

13. Revisión y aprobación de requisitos

La guía de la evidencia solicita contar con documentos o actas de aprobación de requisitos. No se suministró un acta firmada por el cliente o los interesados. Para no atribuir aprobaciones que no se han demostrado, el paquete incluye `docs/ACTA_APROBACION_REQUISITOS_PENDIENTE.md`, una plantilla lista para diligenciar durante la socialización del sistema.

13.1 Registro de aprobación pendiente

Dato	Diligenciar durante la reunión
Fecha, hora y lugar	_____
Cliente/representante	_____
Usuarios participantes	_____
Requisitos revisados	RF-01 a RF-09 y restricciones no funcionales.
Resultado	☐ Aprobado ☐ Aprobado con ajustes ☐ Nueva revisión
Observaciones y compromisos	_____
Firmas	Cliente/interesados: _____ Aprendiz: _____

Estado del documento anexo: plantilla pendiente de diligenciamiento y firma. Debe completarse con participantes reales antes de presentarse como evidencia de aprobación.

13.2 Relación entre requisitos y pruebas

Requisitos	Módulos	Pruebas de referencia
RF-01	Autenticación	Login admin y usuario nuevo, token, credenciales inválidas, acceso sin token.
RF-02	Huéspedes	Alta, búsqueda y consulta.
RF-03	Habitaciones	Consulta y autorización de modificación.
RF-04	Reservas	Alta, noches, detección de conflicto y consulta.
RF-05	Servicios	Catálogo y consumo asociado.
RF-06 / RF-07	Facturación y pagos	Factura, duplicado y abono.
RF-08	Mantenimiento	Alta y actualización de estado.
RF-09	Reportes	Consulta de resumen gerencial.

14. Manual técnico resumido

14.1 Instalación

```
1) Instalar Node.js 18 o superior.
2) Descomprimir el paquete.
3) Abrir Git Bash en ALEJANDRO_RAMIREZ_GA8_EV02.
4) cp .env.example .env
5) Editar JWT_SECRET en .env.
6) node api/server.js
```

14.2 Operación

- Abrir la interfaz en `http://localhost:3000/`.
- Ingresar con una cuenta de demostración local solo para pruebas académicas: `recepcion / 123456` o `admin / 123456`.
- Usar recepción para gestionar huéspedes y reservas; usar administración para operaciones de catálogo y configuración.
- Crear una reserva con huésped y habitación existente; validar disponibilidad antes de confirmar.
- Asociar servicios adicionales a la reserva cuando corresponda.
- Generar factura y registrar pagos sin superar el saldo.
- Consultar mantenimiento y reportes desde sus módulos.

Las credenciales de demostración no deben emplearse en producción y deben cambiarse o eliminarse al desplegar el sistema.

14.3 Comandos de pruebas y construcción

```
node --test tests/*.test.js
# Con el servidor arriba, en otra terminal:
node scripts/api-test.js
# Construir artefacto de distribución:
node scripts/build.js
```

14.4 Resolución de incidentes

Síntoma	Causa posible	Acción
node no se reconoce	PATH no actualizado en terminal	Abrir Git Bash nuevo y ejecutar `node -v`.
Puerto 3000 ocupado	Otro proceso usa el puerto	Cerrar proceso anterior o cambiar PORT.
401 Unauthorized	Token expirado o ausente	Iniciar sesión nuevamente.
403 Forbidden	Rol sin privilegio	Usar perfil autorizado o solicitar permiso.
409 Conflict	Dato duplicado o regla de integridad	Consultar registro existente y corregir datos.

15. Conclusiones

- La entrega articula la interfaz web, los servicios del cliente, la API REST y la persistencia local para soportar el flujo funcional del Hotel Gran Marques.
- La organización por capas y componentes delimita responsabilidades y facilita mantenimiento y evolución del sistema.
- La documentación de entradas y salidas permite identificar qué información requiere cada módulo y qué respuesta debe entregar.
- Las pruebas automatizadas ejecutadas registraron 42 comprobaciones aprobadas: 6 unitarias y 36 de integración, sin fallas.
- La versión académica funciona con almacenamiento JSON y es reproducible localmente; antes de producción debe migrarse a una base de datos transaccional y configurarse HTTPS y gestión de secretos.
- La plantilla de aprobación de requisitos y la URL pública de despliegue quedan pendientes de completar con evidencia real; no se reportan como aprobadas o desplegadas.

16. Referencias

Git. (s. f.). Git documentation: Basic snapshotting. https://git-scm.com/book/en/v2/Appendix-C%3A-Git-Commands-Basic-Snapshotting

Node.js. (s. f.). Node.js API documentation. https://nodejs.org/api/

OWASP Foundation. (s. f.). Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Postman. (s. f.). Postman Learning Center. https://learning.postman.com/

OpenAPI Initiative. (s. f.). OpenAPI Specification. https://spec.openapis.org/oas/latest.html

Servicio Nacional de Aprendizaje [SENA]. (2026). Guía de aprendizaje: Evidencia GA8-220501096-AA1-EV02, Módulos integrados. Material de orientación suministrado en el programa de formación.